Teoria współbieżności

Sprawozdanie – laboratorium 2: semafory i monitory w Javie

Maksymilian Siemek | 12 października 2026

1. Treść zadań

1. Zaimplementować semafor binarny za pomocą metod `wait` i `notify` oraz użyć go do synchronizacji programu *Wyścig* (licznik z laboratorium 1, zadanie 1.1).
2. Pokazać, że do implementacji semafora za pomocą `wait` i `notify` nie wystarczy instrukcja `if`, tylko potrzebna jest pętla `while`. Wyjaśnić teoretycznie dlaczego i potwierdzić eksperymentem. Wskazówka: rozważyć dwie kolejki – czekającą na wejście do monitora obiektu oraz kolejkę związaną z instrukcją `wait` – i ustalić, kto i kiedy jest budzony oraz kiedy następuje wyścig.
3. Zaimplementować semafor licznikowy (ogólny) za pomocą semaforów binarnych. Czy semafor binarny jest szczególnym przypadkiem semafora ogólnego?

2. Argumentacja rozwiązania

Programy napisano w Javie (pakiety `zadanie1`, `zadanie2`, `zadanie3`) i uruchamiano na OpenJDK 25 (Windows). Operacje na semaforze S definiujemy tak jak na stronie laboratorium [1]: `P()` (opuszczenie) – jeśli $S = 1$, to $S := 0$, w przeciwnym razie wstrzymaj wątek; `V()` (podniesienie) – jeśli są wątki wstrzymane na `P()`, to obudź jeden z nich, w przeciwnym razie $S := 1$.

2.1. Semafor binarny i program Wyścig (1)

Semafor przechowuje jedno pole logiczne `state` (`true` – otwarty). Metody są `synchronized`, więc zajmują monitor obiektu semafora. `P()` usypia wątek w `wait()`, dopóki semafor jest zamknięty, a `V()` otwiera semafor i budzi jeden czekający wątek przez `notify()`. Klasa `Counter` z laboratorium 1 chroni teraz każdą operację `value++` i `value--` parą `P()` / `V()`; `V()` jest w bloku `finally`, aby semafor został zwolniony także przy wyjątku:

src/zadanie1/Main.java

```
package zadanie1;

class BinarySemaphore {
    private boolean state;

    public BinarySemaphore() {
        this(true);
    }

    public BinarySemaphore(boolean initialState) {
        this.state = initialState;
    }

    public synchronized void P() throws InterruptedException {
        while (!state) {
            wait();
        }
        state = false;
    }

    public synchronized void V() {
        state = true;
        notify();
    }
}

class Counter {
    private int value = 0;
    private final BinarySemaphore semaphore;

    public Counter(BinarySemaphore semaphore) {
        this.semaphore = semaphore;
    }
```

```
    public void increment() throws InterruptedException {
        semaphore.P();
        try {
            value++;
        } finally {
            semaphore.V();
        }
    }

    public void decrement() throws InterruptedException {
        semaphore.P();
        try {
            value--;
        } finally {
            semaphore.V();
        }
    }

    public int getValue() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        int runs = 20;
        int iterations = 100000;
        int incorrect = 0;

        for (int r = 0; r < runs; r++) {
            Counter counter = new Counter(new BinarySemaphore());

            Thread t1 = new Thread(() -> {
                try {
                    for (int i = 0; i < iterations; i++) counter.increment();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });

            Thread t2 = new Thread(() -> {
                try {
                    for (int i = 0; i < iterations; i++) counter.decrement();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });

            t1.start();
            t2.start();

            t1.join();
            t2.join();

            System.out.println("Uruchomienie " + (r + 1) + ": końcowa wartość licznika = " +
counter.getValue());
            if (counter.getValue() != 0) incorrect++;
        }

        System.out.println("Niepoprawnych wyników: " + incorrect + " / " + runs);
    }
}
```

Semafor zamienia sekcję `value++` / `value--` w sekcję krytyczną – w danym momencie wykonuje ją jeden wątek, więc odczyt, modyfikacja i zapis nie mogą się przeplatać (por. lost update z laboratorium 1). Eksperyment powtórzono 20 razy, po 100 000 operacji na wątek.

2.2. Dlaczego `while`, a nie `if` (2)

Z monitorem każdego obiektu związane są **dwie** kolejki:

- *kolejka wejściowa* (entry set) – wątki chcące wejść do bloku `synchronized`, które czekają na zajęty monitor,

- *kolejka oczekujących* (wait set) – wątki, które wywołały `wait()`: zwolniły monitor i śpią, aż inny wątek wywoła `notify()`.

`notify()` nie przekazuje monitora obudzonemu wątkowi. Przenosi go tylko z wait set do kolejki wejściowej, gdzie musi on konkurować o monitor z wszystkimi innymi chętnymi. Wątek wywołujący `notify()` zwalnia monitor dopiero po wyjściu z metody `synchronized`, a między `notify()` a faktycznym wznowieniem obudzonego wątku inny wątek może zdążyć wejść do monitora. Powstaje wyścig o monitor, a stan semafora może się zmienić, zanim obudzony wątek wróci z `wait()`.

Scenariusz błędu przy `if` (wątki A, B):

krok	zdarzenie	`state`
1	A wykonał `P()` i jest w sekcji krytycznej	`false`
2	B wywołuje `P()`, widzi `false`, usypia w `wait()` (wait set)	`false`
3	A wykonuje `V()`: `state = true`, `notify()` przenosi B do kolejki wejściowej	`true`
4	A opuszcza monitor i znów wywołuje `P()` – monitor jest wolny, `state == true`, więc przechodzi dalej	`false`
5	B dostaje monitor i wraca z `wait()`. Przy `if` warunek **nie jest sprawdzany ponownie** – B ustawia `state = false` i wchodzi do sekcji	`false`

W kroku 5 w sekcji krytycznej są jednocześnie A i B – wzajemne wykluczanie zostało złamane. Przy `while` wątek B po wybudzeniu sprawdza warunek ponownie, widzi `state == false` i wraca do `wait()`. Pętla gwarantuje więc, że warunek jest weryfikowany w chwili, gdy wątek **posiada monitor**, a nie w chwili powiadomienia. Chroni też przed *spurious wakeups* (wybudzeniem bez `notify`), które dopuszcza specyfikacja Javy [3].

Do eksperymentu przygotowano dwie wersje semafora różniące się wyłącznie tą instrukcją (`SemaphoreIf`, `SemaphoreWhile`). Osiem wątków wykonuje po 100 000 razy: `P()`, sekcja krytyczna, `V()`. W sekcji krytycznej zliczamy (licznikiem atomowym), ilu wątków jest w niej jednocześnie, oraz zwiększamy wspólny zwykły licznik `sharedCounter++`:

src/zadanie2/Main.java

```
package zadanie2;

class SemaphoreIf {
    private boolean state = true;

    public synchronized void P() throws InterruptedException {
        if (!state) {          // BŁĄD: warunek nie jest sprawdzany ponownie po obudzeniu
            wait();
        }
        state = false;
    }

    public synchronized void V() {
        state = true;
        notify();
    }
}

class SemaphoreWhile {
    private boolean state = true;

    public synchronized void P() throws InterruptedException {
        while (!state) {       // POPRAWNIE: warunek sprawdzany po każdym obudzeniu
            wait();
        }
        state = false;
    }

    public synchronized void V() {
        state = true;
        notify();
    }
```

```
}

interface Sem {
    void P() throws InterruptedException;

    void V();
}

public class Main {
    static int sharedCounter;

    static void experiment(String name, Sem sem, int threads, int iterations) throws InterruptedException {
        sharedCounter = 0;
        AtomicInteger inside = new AtomicInteger();
        AtomicInteger violations = new AtomicInteger();
        AtomicInteger maxInside = new AtomicInteger();

        Thread[] workers = new Thread[threads];
        for (int t = 0; t < threads; t++) {
            workers[t] = new Thread(() -> {
                try {
                    for (int i = 0; i < iterations; i++) {
                        sem.P();
                        int now = inside.incrementAndGet();
                        if (now > 1) violations.incrementAndGet();
                        maxInside.accumulateAndGet(now, Math::max);
                        sharedCounter++;
                        inside.decrementAndGet();
                        sem.V();
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }

        for (Thread w : workers) w.start();
        for (Thread w : workers) w.join();

        System.out.println(name + ":");
        System.out.println("  max wątków jednocześnie w sekcji krytycznej = " + maxInside.get());
        System.out.println("  naruszeń wzajemnego wykluczania             = " + violations.get());
        System.out.println("  licznik = " + sharedCounter + " (oczekiwano " + (threads * iterations) + ")");
    }

    public static void main(String[] args) throws InterruptedException {
        int threads = 8;
        int iterations = 100000;

        SemaphoreIf semIf = new SemaphoreIf();
        experiment("Semafor z instrukcją if   ", new Sem() {
            public void P() throws InterruptedException { semIf.P(); }
            public void V() { semIf.V(); }
        }, threads, iterations);

        SemaphoreWhile semWhile = new SemaphoreWhile();
        experiment("Semafor z instrukcją while", new Sem() {
            public void P() throws InterruptedException { semWhile.P(); }
            public void V() { semWhile.V(); }
        }, threads, iterations);
    }
}
```

2.3. Semafor licznikowy z semaforów binarnych (3)

Wartość semafora ogólnego (`value`) chroni semafor binarny `mutex`. Wątki, które nie mogą przejść (`value == 0`), usypiają na drugim semaforze binarnym `queue`, a ich liczbę przechowuje `waiting`.

Naiwne rozwiązanie jest błędne:

```
void P() { mutex.P(); value--; if (value < 0) { mutex.V(); queue.P(); } else mutex.V(); }
void V() { mutex.P(); value++; if (value <= 0) queue.V(); mutex.V(); }
```

Semafor binarny nie zlicza podniesień: drugie `queue.V()` wykonane, zanim obudzony wątek zdąży wznowić działanie, nie zwiększa niczego (`state` już jest `true`) i budzenie ginie. Jeden z czekających wątków zostaje uśpiony na stałe. Eksperyment potwierdził to: przy 10 wątkach, semaforze o wartości 3 i po 2000 iteracji w każdym z trzech uruchomień dwa wątki pozostawały zablokowane po 5 sekundach.

Dlatego użyto techniki *przekazywania pałeczki* (passing the baton). Wątek, który wykonuje `V()` (lub kończy `P()`) i widzi czekających, **nie zwalnia** `mutex`, tylko budzi czekającego (`queue.V()`) – ten dziedziczy `mutex`, zmniejsza `waiting`, wykonuje `value--` i dopiero on zwalnia `mutex` albo przekazuje pałeczkę dalej. Dzięki temu `queue.V()` jest wołane tylko wtedy, gdy `queue` ma stan `false`, a stan `value` / `waiting` nigdy nie jest widoczny w stanie pośrednim:

src/zadanie3/Main.java

```java
package zadanie3;

class BinarySemaphore {
    private boolean state;

    public BinarySemaphore(boolean initialState) {
        this.state = initialState;
    }

    public synchronized void P() throws InterruptedException {
        while (!state) {
            wait();
        }
        state = false;
    }

    public synchronized void V() {
        state = true;
        notify();
    }
}

class CountingSemaphore {
    private int value;
    private int waiting = 0;
    private final BinarySemaphore mutex = new BinarySemaphore(true);
    private final BinarySemaphore queue = new BinarySemaphore(false);

    public CountingSemaphore(int initialValue) {
        if (initialValue < 0) throw new IllegalArgumentException("wartość początkowa < 0");
        this.value = initialValue;
    }

    public void P() throws InterruptedException {
        mutex.P();
        if (value == 0) {
            waiting++;
            mutex.V();
            queue.P();          // po obudzeniu wątek dziedziczy mutex (pałeczka)
            waiting--;
        }
        value--;
        release();
    }

    public void V() throws InterruptedException {
        mutex.P();
        value++;
        release();
    }

    // Wywoływane przez wątek posiadający mutex.
    private void release() {
        if (waiting > 0 && value > 0) {
            queue.V();          // przekazanie pałeczki czekającemu wątkowi
        } else {
            mutex.V();
        }
    }
```

```
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        int permits = 3;
        int threads = 10;
        int iterations = 2000;

        CountingSemaphore semaphore = new CountingSemaphore(permits);
        AtomicInteger inside = new AtomicInteger();
        AtomicInteger maxInside = new AtomicInteger();
        AtomicInteger violations = new AtomicInteger();

        Thread[] workers = new Thread[threads];
        for (int t = 0; t < threads; t++) {
            workers[t] = new Thread(() -> {
                try {
                    for (int i = 0; i < iterations; i++) {
                        semaphore.P();
                        int now = inside.incrementAndGet();
                        if (now > permits) violations.incrementAndGet();
                        maxInside.accumulateAndGet(now, Math::max);
                        Thread.yield();
                        inside.decrementAndGet();
                        semaphore.V();
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }

        for (Thread w : workers) w.start();
        for (Thread w : workers) w.join();

        System.out.println("Semafor licznikowy, wartość początkowa = " + permits);
        System.out.println("Wątków: " + threads + ", iteracji na wątek: " + iterations);
        System.out.println("Maksymalnie jednocześnie w sekcji krytycznej: " + maxInside.get());
        System.out.println("Naruszeń limitu: " + violations.get());

        // Semafor ogólny z wartością 1 używany jako mutex (zachowuje się jak binarny)
        CountingSemaphore mutexLike = new CountingSemaphore(1);
        int[] counter = {0};
        Thread[] incs = new Thread[4];
        for (int t = 0; t < incs.length; t++) {
            incs[t] = new Thread(() -> {
                try {
                    for (int i = 0; i < 50000; i++) {
                        mutexLike.P();
                        counter[0]++;
                        mutexLike.V();
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }
        for (Thread w : incs) w.start();
        for (Thread w : incs) w.join();
        System.out.println("Semafor ogólny (1) jako mutex: licznik = " + counter[0] + " (oczekiwano
200000)");
    }
}
```

Program testuje semafor o wartości 3 przy 10 wątkach (po 2000 wejść) – liczy maksymalną liczbę wątków w sekcji krytycznej. Dodatkowo semafor o wartości 1 służy jako mutex dla zwykłego licznika.

Czy semafor binarny jest szczególnym przypadkiem ogólnego? W sensie użycia – tak: semafor ogólny z wartością początkową 1, używany poprawnie (każde `V()` poprzedzone własnym `P()`), nigdy nie przekracza wartości 1 i zachowuje się jak binarny (tak działa mutex w ostatnim teście). Ściśle jednak operacja `V()` jest inna: w binarnym przy `state == 1` jest bezskuteczna (nasycenie), a w ogólnym zwiększa wartość do 2, 3, … Semafor

binarny to więc semafor ogólny z wartością ograniczoną do $\{0, 1\}$, a nie dokładnie ten sam obiekt. Oba mają tę samą siłę wyrazu, bo – jak pokazano – ogólny można zbudować z binarnych.

3. Uzyskane wartości

Zadanie 1 – 20 uruchomień z semaforem binarnym (po 100 000 operacji na wątek): w każdym końcowa wartość licznika wynosiła **0** (0 niepoprawnych wyników na 20). Bez semafora (laboratorium 1) wyniki były losowe, np. 5498, 315, −333.

Zadanie 2 – pięć uruchomień eksperymentu (8 wątków × 100 000 wejść, oczekiwany licznik 800 000):

uruchomienie	`if`			`while`		
	max w sekcji	naruszenia	licznik	max w sekcji	naruszenia	licznik
1	4	1389	799 977	1	0	800 000
2	3	1248	799 983	1	0	800 000
3	4	3351	799 791	1	0	800 000
4	4	2195	799 974	1	0	800 000
5	3	2485	799 982	1	0	800 000

Zadanie 3 – semafor o wartości 3: w trzech uruchomieniach maksymalna liczba wątków w sekcji krytycznej wynosiła **3**, a liczba naruszeń limitu **0**. Semafor ogólny o wartości 1 użyty jako mutex dał licznik **200 000** ze 200 000 oczekiwanych.

4. Weryfikacja poprawności

1. Wynik 0 w każdym z 20 uruchomień zgadza się z oczekiwaniem: obie sekcje krytyczne wykluczają się, więc liczba zwiększeń i zmniejszeń jest równa. Wzajemne wykluczanie wynika wprost z definicji semafora: gdy `state == false`, żaden inny wątek nie przejdzie przez `P()`, aż właściciel wykona `V()`. Przejście przez `while` odbywa się pod monitorem, więc sprawdzenie i zmiana `state` są atomowe.

2. Wersja z `if` w każdym uruchomieniu łamie wzajemne wykluczanie: do 4 wątków jednocześnie w sekcji, od 1248 do 3351 naruszeń, a licznik jest mniejszy od 800 000 (utracone aktualizacje, jak w laboratorium 1). Wersja z `while` we wszystkich uruchomieniach daje maksymalnie 1 wątek w sekcji, 0 naruszeń i poprawny licznik. Liczby naruszeń zależą od przeplotu i różnią się między uruchomieniami, ale ich występowanie jest powtarzalne. Wynik zgadza się ze scenariuszem z tabeli w p. 2.2. Warto zaznaczyć, że eksperyment nie dowodzi poprawności wersji z `while` (brak błędu w próbie nie jest dowodem), lecz argument teoretyczny tak: warunek jest sprawdzany pod monitorem bezpośrednio przed zmianą stanu.

3. Poprawność semafora licznikowego wynika z niezmiennika: value ≥ 0, a wątki w `queue` są tylko wtedy, gdy `value == 0`. Pałeczka (`mutex`) jest zawsze w posiadaniu dokładnie jednego wątku albo leży odłożona, więc `value` i `waiting` zmienia tylko ten, kto ją ma. `queue.V()` wykonywane jest tylko wtedy, gdy `waiting > 0` i `value > 0`, a kolejne `queue.V()` dopiero po tym, jak obudzony wątek wznowi pracę z pałeczką – nie ma więc zgubionych budzeń. Eksperyment potwierdza limit (maks. 3 wątki, 0 naruszeń) i brak zakleszczenia (programy zawsze się kończą). W teście z `mutex` licznik wyniósł 200 000.

5. Wnioski

- **Semafor binarny na `wait` / `notify` daje wzajemne wykluczanie.** Sekcja krytyczna chroniona semaforem eliminuje wyścig z laboratorium 1 – wynik zawsze wynosi 0. W przeciwieństwie do zamka Petersona działa dla dowolnej liczby wątków i nie wymaga aktywnego czekania.
- **Obudzenie przez `notify` nie oznacza, że warunek nadal zachodzi.** (Brinch Hansen [2] krytykuje monitory Javy m.in. za brak kontroli takich zależności czasowych w czasie kompilacji i wykonania.) Obudzony wątek musi jeszcze zdobyć monitor, a w międzyczasie inny wątek może zmienić stan. Dlatego `wait()` należy zawsze wywoływać w pętli sprawdzającej warunek (`while`), nie w `if`.
- **Błąd z `if` jest rzadki w pojedynczym wejściu, ale pewny w skali.** Dopiero setki tysięcy wejść ośmiu wątków ujawniają tysiące naruszeń. Samo „działa u mnie” nie jest argumentem za poprawnością programu współbieżnego.

- **Semafor binarny nie zlicza.** Zbudowanie z niego semafora ogólnego wymaga ostrożności: naiwna implementacja gubi budzenia i zakleszcza wątki. Przekazywanie pałeczki rozwiązuje problem kosztem trudniejszego rozumowania o tym, kto aktualnie posiada `mutex`.
- **Semafor binarny jest szczególnym przypadkiem ogólnego tylko przy poprawnym użyciu.** Różnią się zachowaniem `V()` przy wartości 1 (nasycenie vs zwiększenie), ale oba mają taką samą siłę wyrazu.

6. Bibliografia

[1] Strona laboratorium 2: *Teoria współbieżności – Laboratorium 2*, AGH. https://home.agh.edu.pl/~funika/tw/lab2/

[2] P. Brinch Hansen, *Java's Insecure Parallelism*, ACM SIGPLAN Notices, kwiecień 1999, s. 38–45.

[3] J. Gosling i in., *The Java Language Specification, Java SE 25 Edition*, rozdz. 17 „Threads and Locks", Oracle. https://docs.oracle.com/javase/specs/